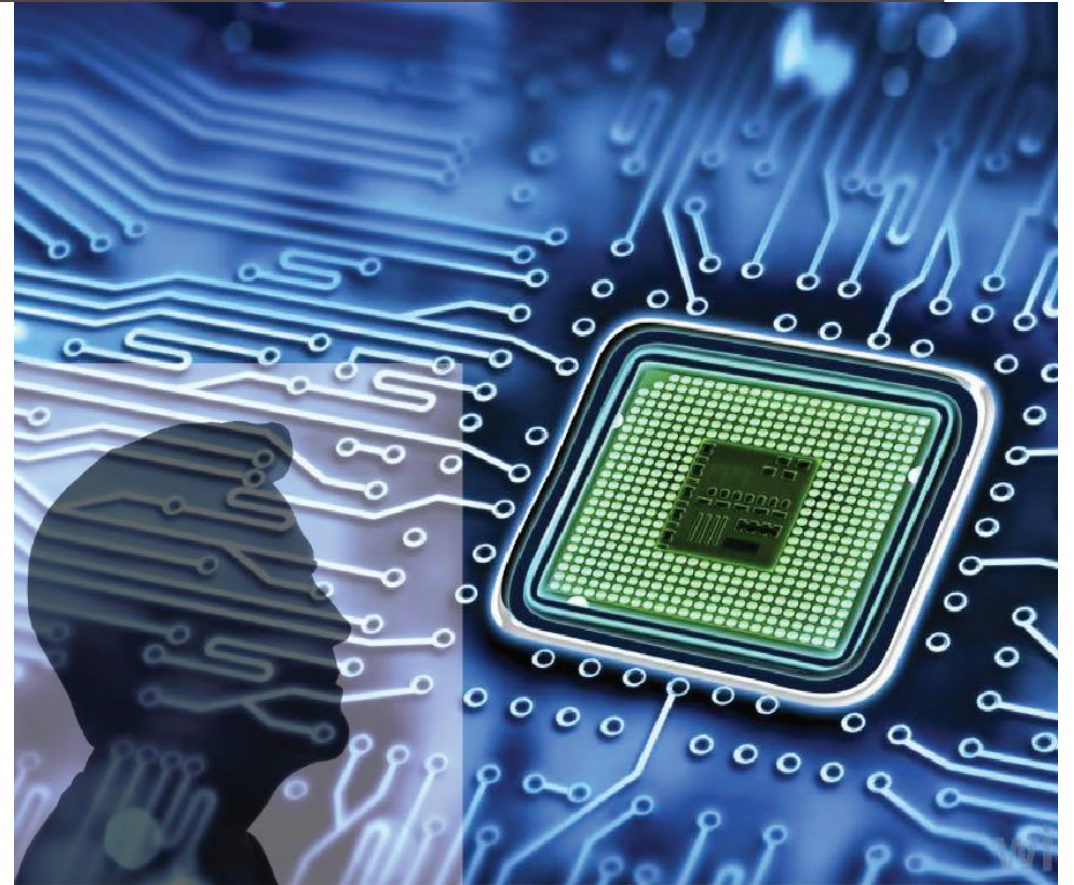


VLSI DESIGN FLOW

Week-3, Lecture-1 Introduction

Sneh Saurabh
28th August, 2024

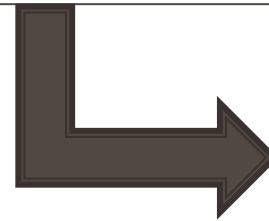


VDF: Revision/Announcements

- Project Topics



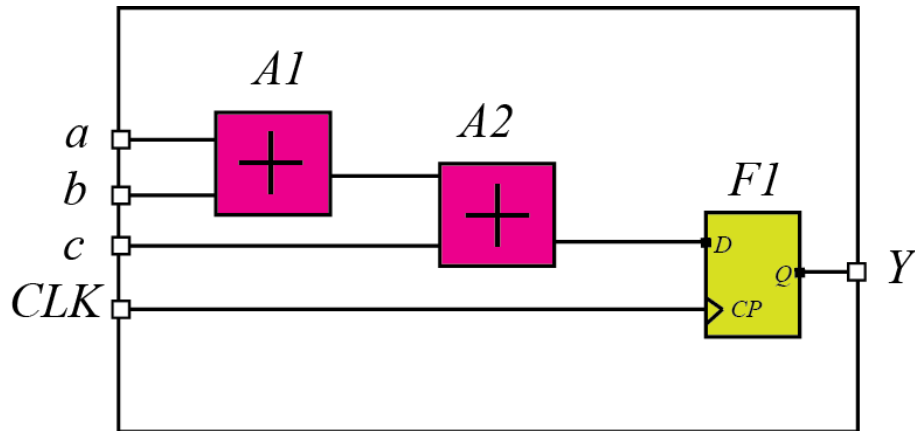
Pre-RTL Methodologies



Behavior Synthesis

Behavioral Synthesis: Illustration (1)

- Algorithmic behavior: $Y = a + b + c$. Let us consider its behavior synthesis.
- Cost metrics: circuit elements used, latency, and maximum delay of combination path.



Resources

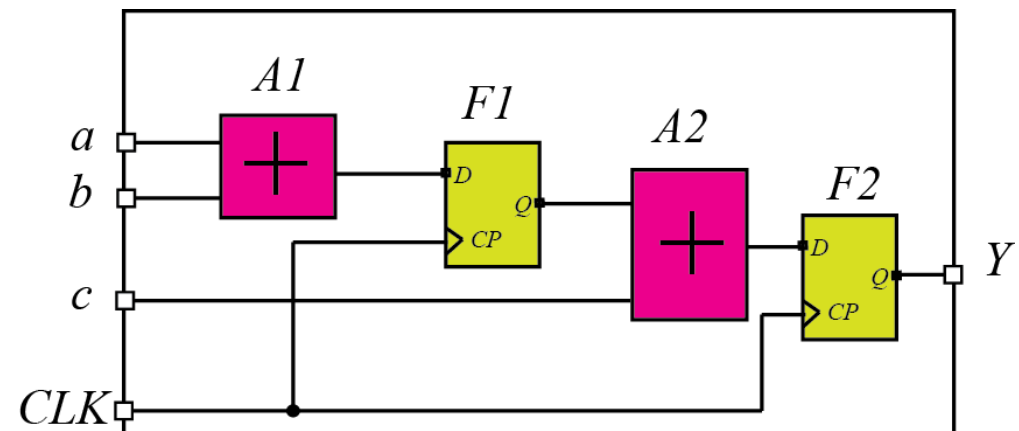
- 2 Adders (+) and 1 Register

Latency

- 1 clock cycle

Worst Delay

- Delay of 2 Adders



Resources

- 2 Adders (+) and 2 Registers

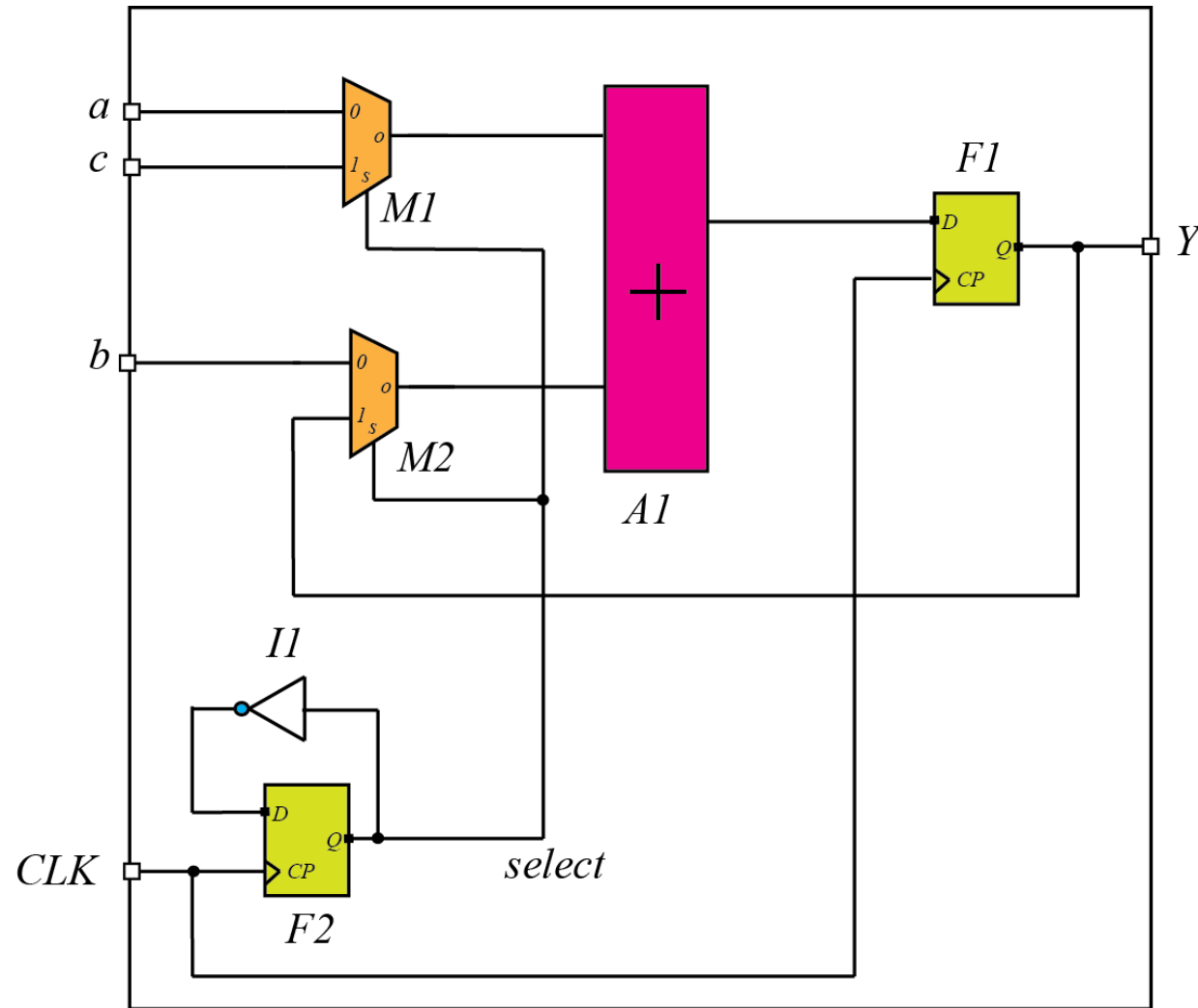
Latency

- 2 clock cycle

Worst Delay

- Delay of 1 Adders

Behavioral Synthesis: Illustration (2)



- Adder is used in first cycle to compute ($Y = a + b$) and is used in the next cycle to compute ($Y = c + Y$)

- Inputs to adders are controlled by multiplexers
- Multiplexers get “select” signal from the control circuitry

Resources

- 1 Adders (+), 2 Register, 2 Multiplexer and 1 Inverter

Latency

- 2 clock cycle

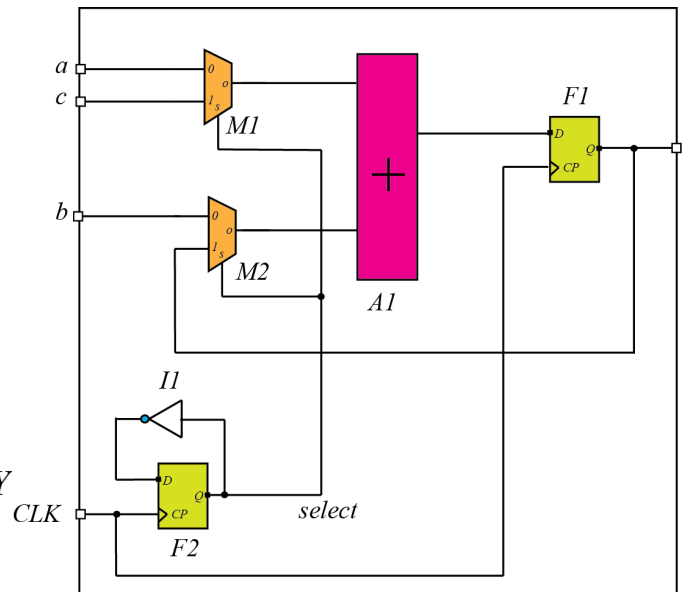
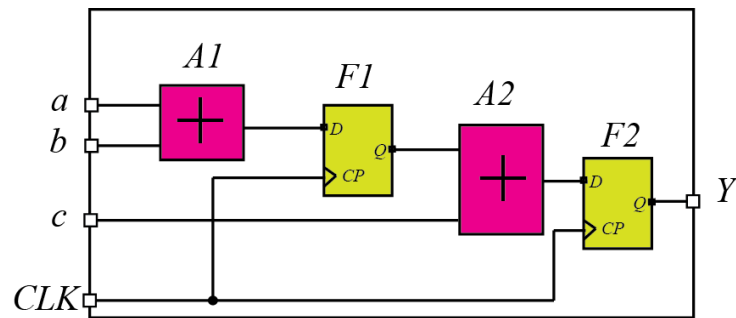
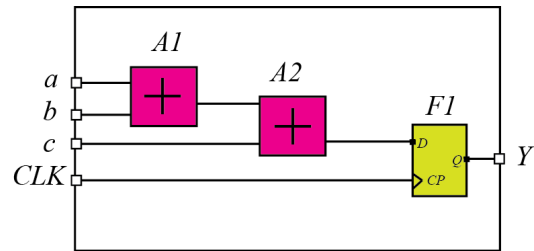
Worst Delay

- Delay of 1 Adder + Multiplexer

Behavioral Synthesis: Evaluating Trade-offs

	Area (μm^2)	Delay (ns)
Inverter	1	1
Multiplexer	6	10
Adder	200	100
Flip-flop	12	0

Let us compute the area, latency and critical path delay for the three implementations considered.



	RTL-1	RTL-2	RTL-3
Area	412	424	237
Latency	1 cycle	2 cycle	2 cycle
Delay	200	100	110

Behavioral Synthesis: Untimed to Timed Behavior

Algorithmic behavior: $Y = a + b + c$

Timed Behavior

- Three different “timed” implementation illustrated
- There can be several other implementations
- Behavior synthesis tool will choose the best possible implementation satisfying the constraints

Trade offs

- Behavior synthesis tool will trade off one FoM to improve other FoM

	RTL-1	RTL-2	RTL-3
Area	412	424	237
Latency	1 cycle	2 cycle	2 cycle
Delay	200	100	110

Which of the RTL will be generated when:

- Area is to be minimized?
- Latency is to be minimized?
- Clock Frequency is to be maximized?

Behavioral Synthesis: Merits and Challenges

Merits:

- Behavioral synthesis allows automatic exploration of different possible implementations : more exhaustive than handwritten RTL
- Reduces design effort
- Without behavior synthesis, RTL will be handwritten which has greater chance of introducing errors

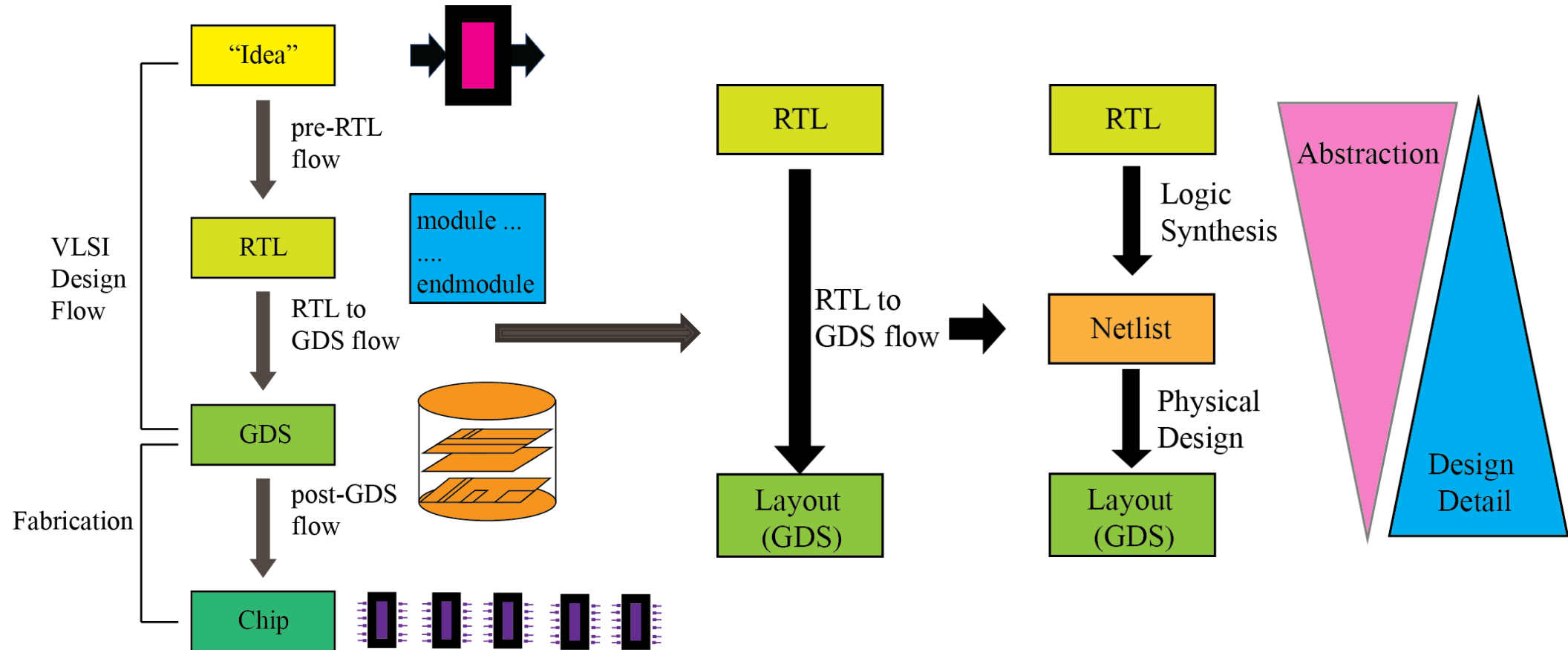
- Behavioral Synthesis must result in a better QoR than hand-written RTL
 - This may not always be possible, manually tweaking generated RTL can improve QoR

- Behavioral Synthesis tool “estimates” timing and area
 - Down the flow the QoR may degrade due to other metrics such as congestion not taken into account during Behavioral Synthesis

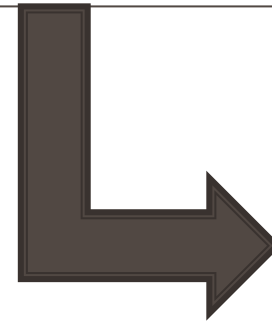
- Verification of Generated RTL
 - Tools not yet developed
 - Equivalence checking

Overview of RTL to GDS Flow

VLSI Design Flow: RTL to GDS Flow



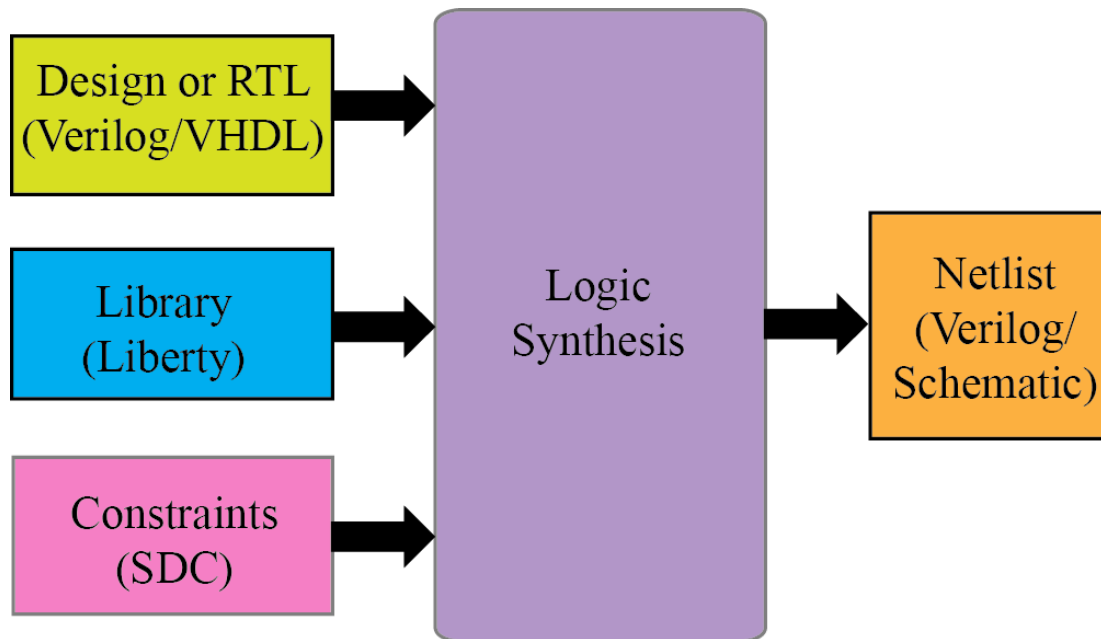
Overview of RTL to GDS Flow



Logic Synthesis

RTL to GDS flow: Logic Synthesis (1)

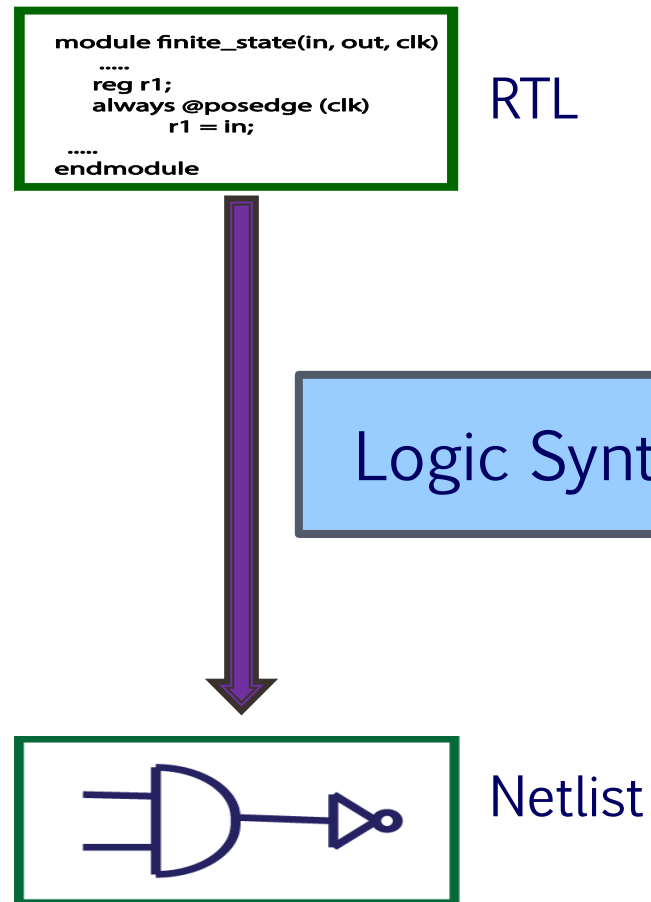
- **Logic Synthesis:** process by which RTL is converted to an equivalent circuit as interconnection of a logic gates



- **RTL:** given design
- **Library:** standard cells and macros
- **Constraints:** design goals, expected timing behavior, environment

- **Netlist:**
 - Interconnection of logic gates
 - Usually represented using Verilog constructs

RTL to GDS flow: Logic Synthesis (2)



- Matured technology
- Industry Standard Tool
 - *Design Compiler*. from Synopsys
 - *Genus* from Cadence

Logic Synthesis: Illustration

```
module top(a, b, clk, select,  
out);
```

```
input a, b, clk, select;
```

```
output out;
```

```
reg out;
```

```
wire y;
```

```
assign y = (select) ? b : a;
```

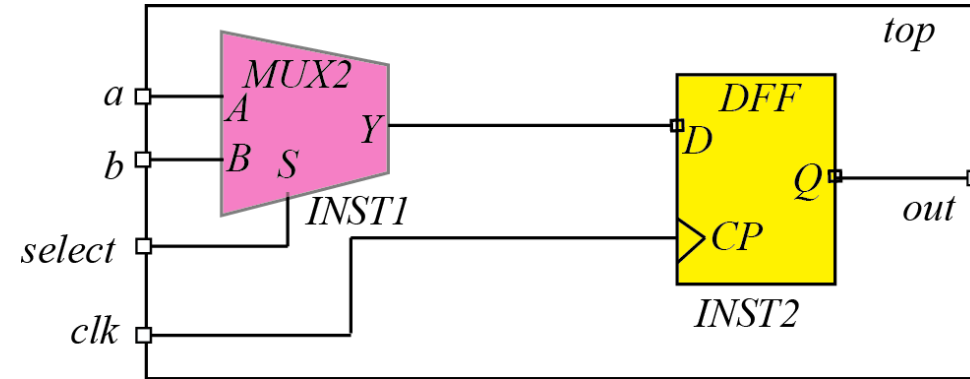
```
always @(posedge clk)
```

```
begin
```

```
    out <= y;
```

```
end
```

```
endmodule
```



```
module top(a, b, clk, select, out);
```

```
input a, b, clk, select;
```

```
output out;
```

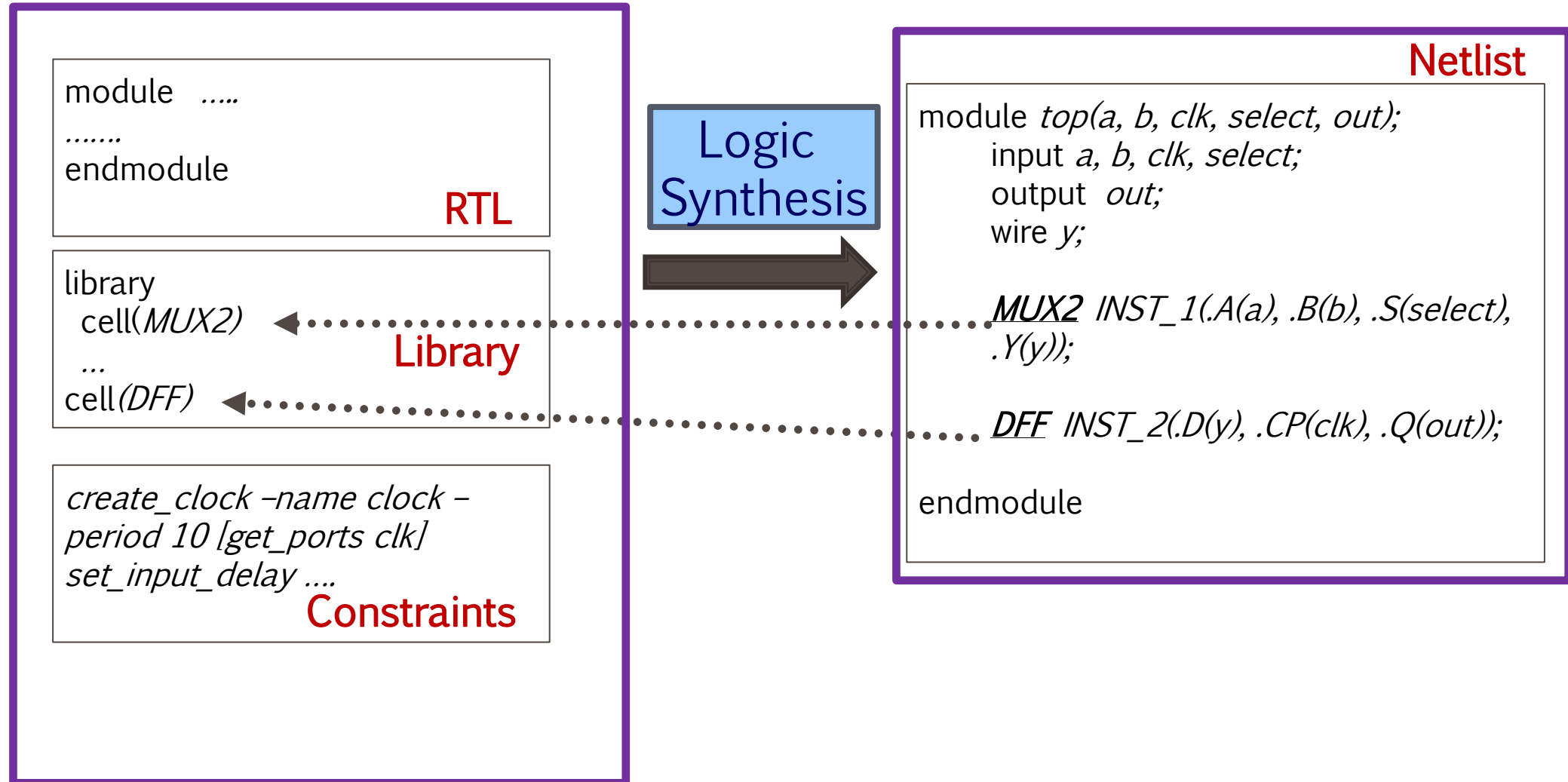
```
wire y;
```

```
MUX2 INST1(.A(a), .B(b), .S(select), .Y(y));
```

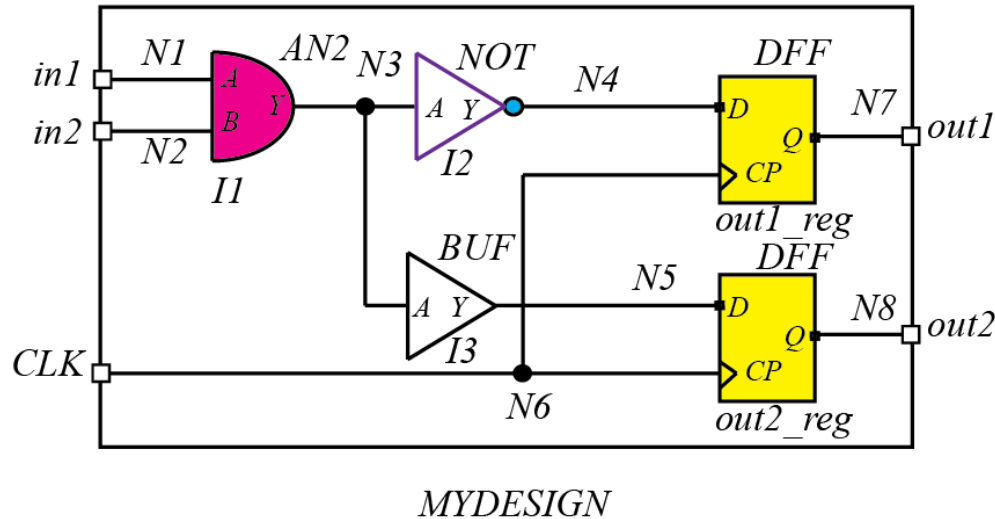
```
DFF INST2(.D(y), .CP(clk), .Q(out));
```

```
endmodule
```

Logic Synthesis: Inputs and Outputs



IC Design: Some Common Terminology (1)



```
library...  
  cell(AN2)...  
    pin(A)...  
    pin(B)...  
    pin(Y)...  
...  
  cell(NOT)...  
  
  cell(BUF)...  
  
  cell(DFF)...
```

Design: Top level entity that represents the circuit.

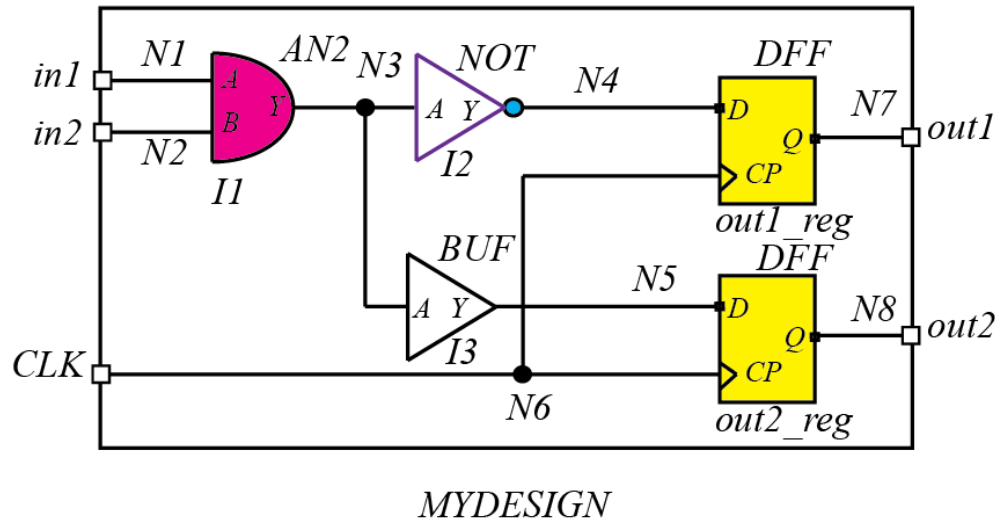
E.g.: *MYDESIGN*

Ports: The interfaces of the *Design* through which it communicates with the external world.

E.g.: *in1*, *in2*, *CLK*, *out1*, *out2*

- **Input Ports:** Signals going inside the design. E.g.: *in1*, *in2*, *CLK*
- **Output Ports:** Signals going outside from the design. E.g. *out1*, *out2*

IC Design: Some Common Terminology (2)



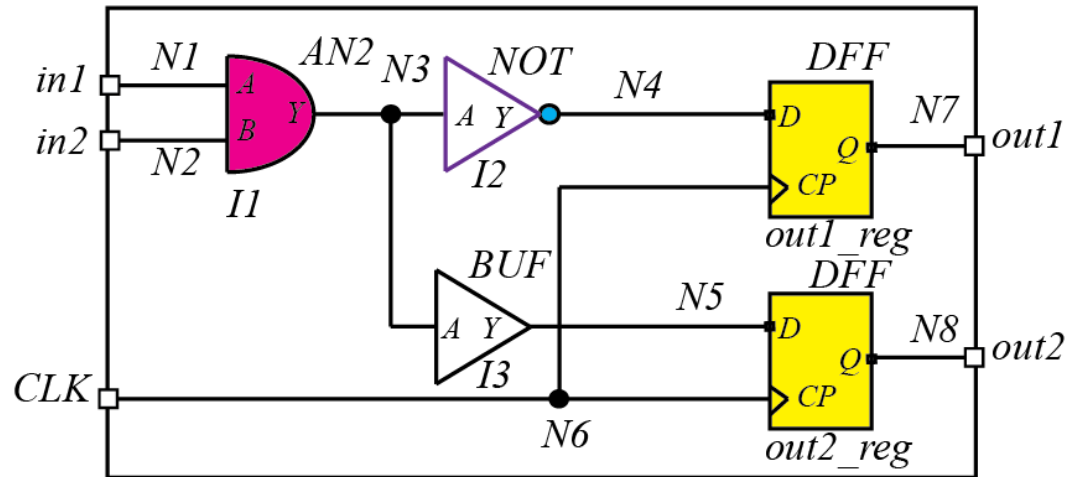
library...

```
cell(AN2)...  
    pin(A)...  
    pin(B)...  
    pin(Y)...  
  
...  
cell(NOT)...  
  
cell(BUF)...  
  
cell(DFF)...
```

- **Cells:** basic entity delivering combinational or sequential function contained in libraries.
E.g.: *AN2*, *NOT*, *BUF*, *DFF*
- Design is composed of multiple *cells* connected together

- **Instances:** cells when used inside a design are called *instances*.
E.g.: *I1*, *I2*, *I3*, *out1_reg*, *out2_reg*.
- Using a cell in a design is called *instantiation*.
- The same cell can be instantiated multiple times. E.g.: *out1_reg* and *out2_reg* are instances of the same cell *DFF*.

IC Design: Some Common Terminology (3)



MYDESIGN

library...

cell(AN2)...
pin(A)...
pin(B)...
pin(Y)...

...
cell(NOT)...

cell(BUF)...

cell(DFF)...

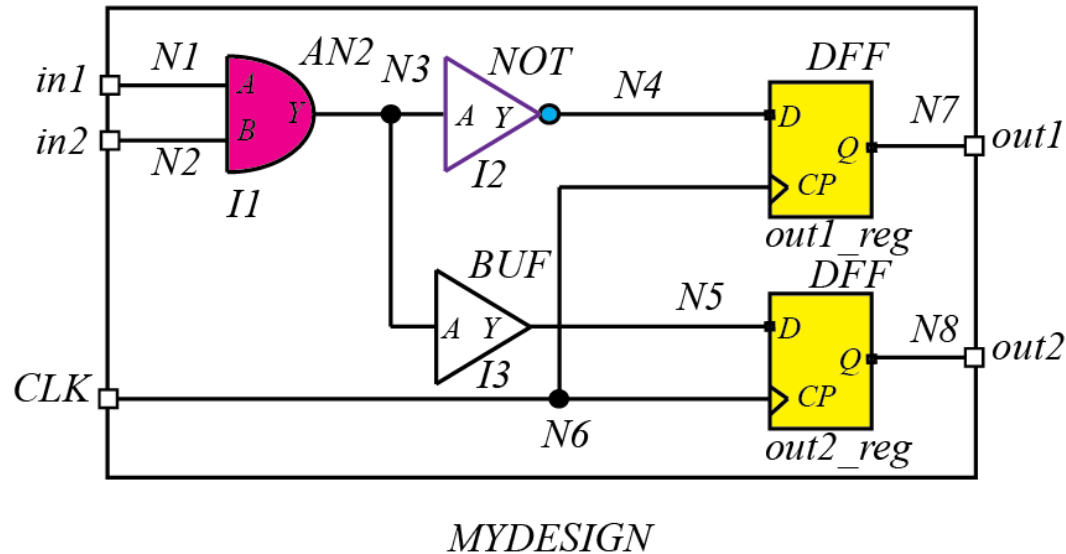
- **Input pin:** that brings signal into the cell/instance. E.g.: *A* and *B*
- **Output pin:** that brings signal out from the cell/instance. E.g.: *Y*

- **Instance pin name:** typically specified as combination of instance name and pin name separated by */*
- E.g.: *I1/A*, *I1/B*, *out1_reg/Q*

- **Pin:** An interface of a library cell or instance through which it communicates with the other components is called a pin.
- E.g.: *A*, *B*, *Y* are the pins of the cell *AN2* and the instance *I1*

- **Library Pin and Instance Pin** (if we want to be explicit)
- Often apparent from the context

IC Design: Some Common Terminology (4)



library...

```
cell(AN2)...  
  pin(A)...  
  pin(B)...  
  pin(Y)...
```

...
cell(*NOT*)...

cell(*BUF*)...

cell(*DFF*)...

- **Net:** The wire that connects different instances and ports is called a net.
E.g.: *N1*, *N2*, *N3*, ...*N8*

Netlist: Attributes

Problem:

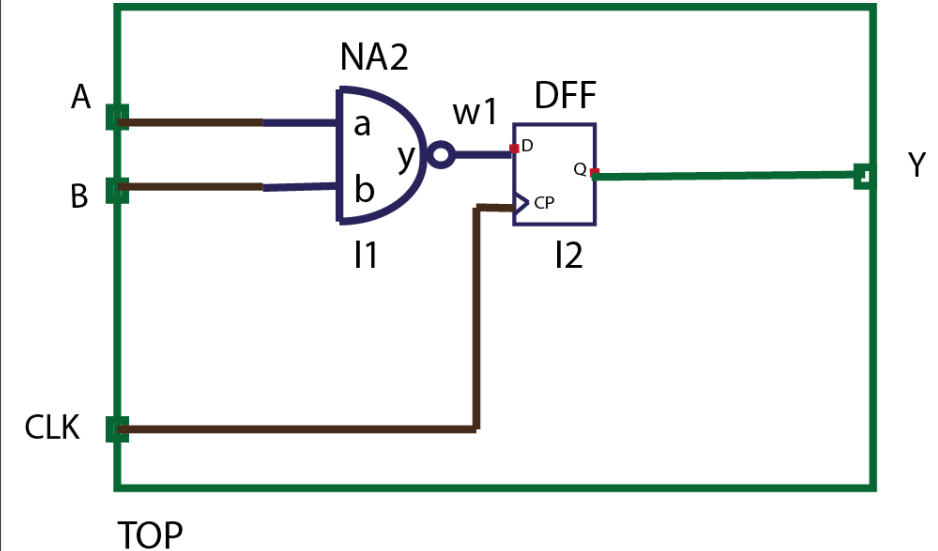
Represent the schematic shown as a Verilog netlist.
Design Name is: TOP

Port Names are: A, B, CLK, Y

Net Names: A, B, CLK, Y, w1

Instance Names are: I1 and I2

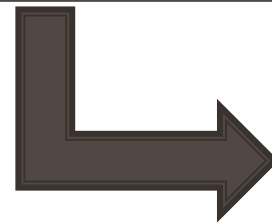
Name of cells in library are: NA2 and DFF



```
module TOP(A, B, CLK, Y);
input A, B, CLK;
output Y;
wire w1;

NA2 I1(.a(A), .b(B), .y(w1));
DFF I2(.D(w1), .CP(CLK), .Q(Y));
endmodule
```

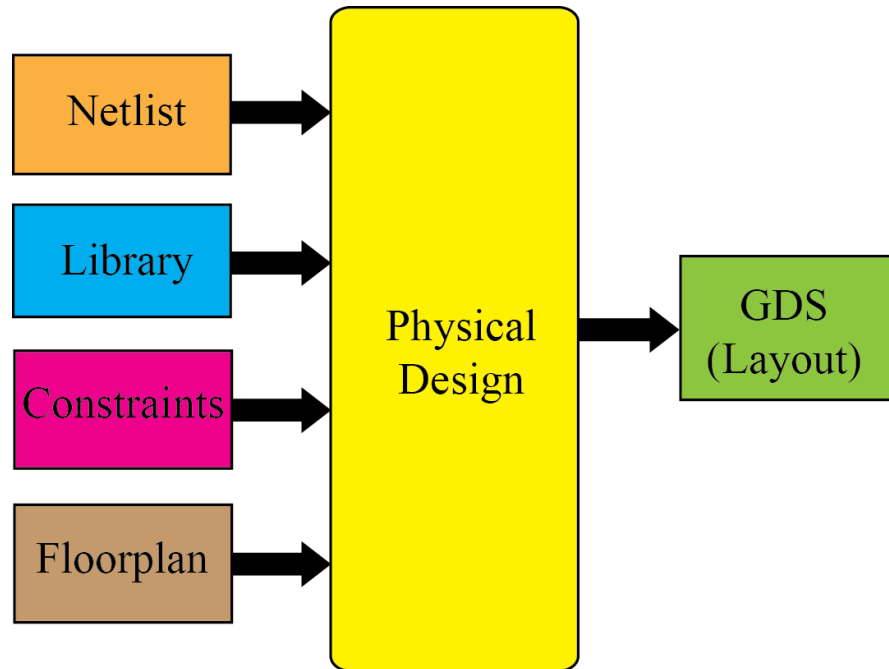
Overview of RTL to GDS Flow



Physical Design

VLSI Design Flow: Physical Design

Physical Design: Process by which a design in the form of a netlist is converted to an equivalent design in the form of layout or GDS (geometrical patterns of masks)



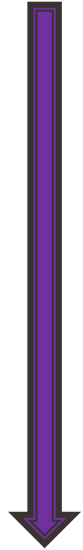
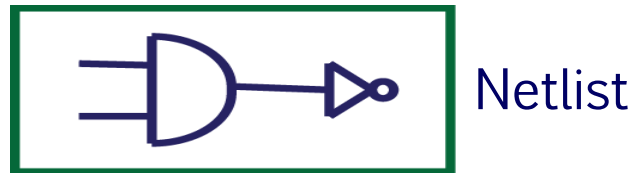
Netlist: input design (output of logic synthesis)

Library: standard cells and macros (similar as in logic synthesis), additional abstract physical information of cells and technology-specific information

Constraints: design goals, expected timing behavior, environment (similar as in logic synthesis)

Floorplan: designer's intent about the physical design (size and shape of die, predefined placement information)

VLSI Design Flow: Physical Design



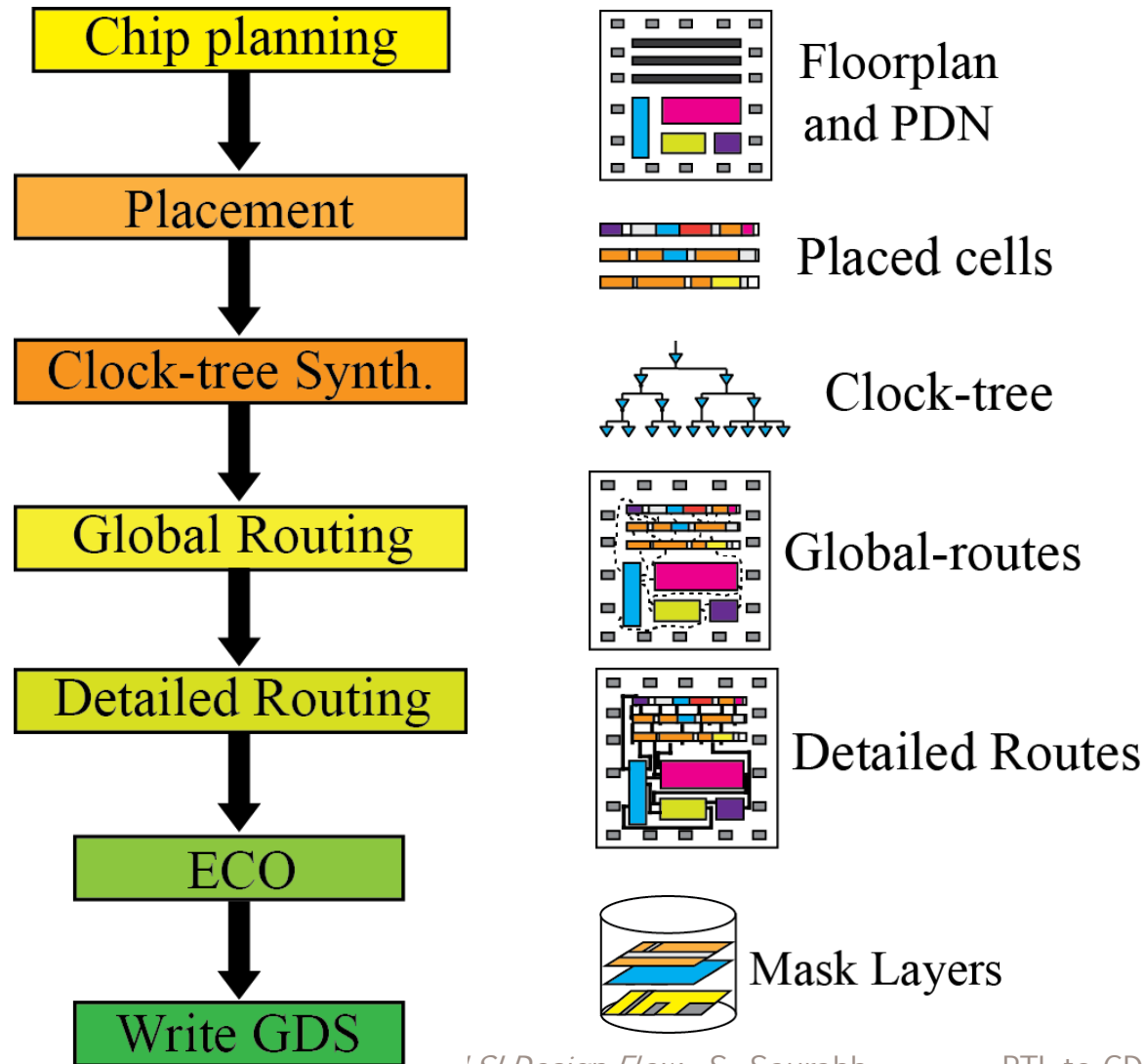
Physical Design



- Input:
 - Netlist, Library, Constraints, Floorplan
- Output
 - GDS: patterns for different layers of mask
- Physical Design done in several distinct steps

- Tools used for Physical Designs are:
 - **IC Compiler**: from Synopsys
 - **Innovus**: from Cadence

Physical Design: Major Tasks



References

- S. Saurabh, “Introduction to VLSI Design Flow”. Cambridge: *Cambridge University Press*, 2023.